Trabalho 2 (T2): Mestre/escravo com MPI

Unidade 04 | Grupos de até 3 integrantes

O trabalho tem quatro momentos: a apresentação deste enunciado, duas aulas de acompanhamento em laboratório, a entrega no Moodle (véspera da apresentação) e a **apresentação para a turma**. As datas estão no cronograma do Moodle. Junto deste enunciado está publicado o **feedback geral do T1**: os problemas listados lá valem como critério de correção aqui.

---

1. Objetivo

Implementar em C e MPI uma versão paralela de uma aplicação seguindo o modelo **mestre/escravo**, executá-la no cluster Atlântica do LAD e **analisar o desempenho** obtido. Como no T1, o foco é a análise: explicar o que causou os números, não repetir em texto o que o gráfico já mostra.

2. O modelo exigido

O mestre (processo 0) guarda o **saco de trabalho**: um conjunto de tarefas independentes (por exemplo, linhas de uma matriz ou posições de um vetor). Os escravos processam as tarefas e devolvem os resultados. A implementação precisa atender a todos os requisitos abaixo:

1. **A iniciativa é do escravo.** Ao começar, cada escravo pede trabalho ao mestre; cada resultado devolvido vale também como pedido da próxima tarefa.
2. **O mestre recebe por ordem de chegada** (`MPI_ANY_SOURCE`), atendendo primeiro quem ficar pronto primeiro.
3. **A ordem original do saco é mantida:** cada resultado volta para a posição de origem.
4. **Saco vazio encerra os escravos** com uma mensagem de término, distinguida por tag.
5. **Funciona com qualquer número de processos** (a partir de 2) e qualquer número de tarefas, sem exigir que um seja múltiplo do outro.
6. **A granularidade é um parâmetro:** quantas unidades de trabalho vão em cada mensagem é lido da linha de comando.

3. Escolha do problema

O ponto de partida é **o problema do T1**, se ele se encaixar no modelo: deve ser possível quebrá-lo em muitas tarefas independentes, mais numerosas que os escravos, cujo custo de processamento seja bem maior que o de enviá-las e recebê-las.

Se o problema do T1 não se encaixar, escolha um problema novo. A escolha precisa ser validada pelo professor no primeiro acompanhamento.

Ferramentas de IA estão liberadas nas mesmas condições do T1: o grupo deve saber explicar o código linha a linha, e o relatório deve dizer como a ferramenta foi usada. Código de terceiros deve ter a origem citada.

4. Ambiente e regras de medição

Todas as medições, inclusive a sequencial, são feitas **no cluster Atlântica**. Cada nó tem 2 processadores de 8 núcleos físicos, com Hyper-Threading (HT): 16 núcleos físicos e 32 threads de hardware por nó. Confirme com `lscpu` e registre no relatório.

1. **Nós exclusivos.** Rode com `--exclusive`, sempre nos mesmos **2 nós**, e prefira jobs em lote (`sbatch`) a execuções interativas. Diga como os processos foram distribuídos entre os nós.
2. **T_1 é o da melhor versão sequencial** (sem MPI), compilada com as mesmas flags de otimização da paralela e executada em um nó do cluster Atlântica. Dimensione a entrada de referência para que T_1 fique **entre 30 e 60 segundos**.

3. **Região medida:** com `MPI_Wtime()` no mestre, do início da distribuição até a chegada do último resultado. A geração ou leitura da entrada fica de fora nas duas versões, com os mesmos limites.
4. **Equivalência registrada:** toda execução imprime uma grandeza verificável do resultado (checksum, contagem, soma), que deve coincidir com a da versão sequencial.
5. **10 repetições** por configuração; reporte mediana, mínimo e máximo em todas as tabelas.

5. Medições exigidas

Escalabilidade forte. Com a entrada de referência fixa, rode com P = 2, 4, 8, 16 e 32 processos (até 1 por núcleo físico) e com 64 (HT). Calcule speed-up $S = T_1/T_P$ e eficiência $E = S/P$. O mestre ocupa um núcleo e conta em P.

Escalabilidade fraca. Partindo da mesma entrada de referência, aumente o **número de tarefas** na proporção de P, mantendo o custo de cada tarefa, na mesma série de processos. Calcule $E_{\text{fraca}} = T_1/T_P$, em que T_P é o tempo com P processos e trabalho P vezes maior. Mostre com uma medida de trabalho (tarefas, iterações, operações) que ele cresceu na proporção certa.

Granularidade e balanceamento. Na configuração com 32 processos, (a) meça o tempo de processar uma tarefa e o de uma ida e volta de mensagem do mesmo tamanho, (b) compare a granularidade escolhida com pelo menos uma bem menor e uma bem maior, e (c) registre por escravo o número de tarefas processadas e o tempo ocupado.

Apresentação. Uma tabela por experimento e, no mínimo, um gráfico de speed-up da forte com a reta ideal e um de eficiência com as curvas da forte e da fraca. Não é preciso plotar o tempo.

6. O relatório

No máximo duas páginas em PDF, incluindo referências: A4, margens de pelo menos 1,5 cm, fonte 10 ou 11, uma ou duas colunas. Cabeçalho compacto com integrantes e **link do repositório**; sem capa, sumário ou apêndices. Toda figura e tabela tem legenda e é referenciada no texto. A **análise ocupa a maior parte** do relatório e cobre:

1. **Modelagem:** o que é uma tarefa, como o saco é montado, quais tags existem, como a ordem é mantida e como os escravos terminam, referenciando o código por arquivo e linha.
2. **Ambiente e metodologia:** nós, distribuição dos processos, compilador e flags, região medida, entradas e regra de crescimento da fraca.
3. **Granularidade e balanceamento:** a razão entre computação e comunicação de uma tarefa, o efeito medido de mudar a granularidade e como a carga se distribuiu entre os escravos, inclusive no fim do saco.
4. **Análise dos resultados:** as causas das curvas. Onde o speed-up satura e por quê: custo de comunicação dentro do nó e entre os nós, mestre como gargalo, núcleo ocupado pelo mestre, desbalanceamento no fim do saco, efeito do HT. Se a fraca se sustenta melhor que a forte. Cada causa apontada deve estar apoiada em uma medição.
5. **Uso de ferramentas de IA**, se houve.

7. Código e entrega

O código vai em um repositório git **público** (link no cabeçalho) ou em um `.zip` enviado no Moodle junto do PDF. A entrega precisa permitir **reproduzir as tabelas e os gráficos**: fonte das versões sequencial e paralela, `Makefile`, entradas ou gerador, os scripts `sbatch`, as saídas do Slurm com os tempos brutos de todas as execuções e o que for preciso para recalcular as métricas. Um `README` curto traz os comandos exatos. Nomes claros e comentários nos pontos não óbvios (o significado de cada tag, por que aquela granularidade) contam na avaliação.

No Moodle, até a data do cronograma, um envio por grupo.

8. Apresentação

Em sala, para a turma, em **no máximo 10 minutos**, priorizando a modelagem e a análise. Cada integrante precisa estar presente e participar; quem faltar ou não apresentar recebe zero, mesmo que o grupo tenha entregado. Qualquer integrante pode ser questionado sobre qualquer chamada MPI do código e qualquer número do relatório.

9. Avaliação

Cada item é avaliado em quatro níveis: atingiu plenamente, parcialmente, quase não atingiu, não atingiu. A presença na apresentação é condição individual e antecede estes critérios.

#	Item
1	O relatório está bem escrito e no formato solicitado
2	O problema é adequado ao modelo mestre/escravo e tem complexidade compatível
3	A implementação atende aos seis requisitos da seção 2
4	As versões sequencial e paralela produzem resultados equivalentes, e isso está registrado
5	A entrega permite reproduzir os resultados apresentados
6	O protocolo de medição foi seguido e as duas escalabilidades foram medidas e plotadas
7	A granularidade foi justificada por medição e o balanceamento entre os escravos foi analisado
8	A análise explica as causas dos resultados, incluindo comunicação, mestre, distribuição entre nós e HT
9	Participação do grupo nas aulas de acompanhamento
10	Na apresentação, cada integrante demonstrou domínio do código, dos resultados e respondeu às perguntas

Os trabalhos T3 e T4 usam o mesmo cluster e as mesmas regras da seção 4: os scripts de medição feitos agora serão reaproveitados.